Induction Game

Shahrukh Mohiuddin

Game concept

Game Master

Your role:
You (the AI) will propose inputs for a hidden function so you can infer it.

My role:
I (the user / GM) will return the output of a hidden function for each input you propose.

Goal:
Infer the exact function:
$**SIGNATURE_OF_CURRENT_FUNCTION**$

Context:
Category: $CATEGORY$.
This function takes NUM_PARAMS parameter(s) with names and types:
$PARAM_LIST$
(Format shown below.)
.
.
.

Output: 0

Player 1

(x: int, y: int) -> int

Context:
Category: math.
This function takes 2 parameter(s) with names and types:
x: int, y: int
(Format shown below.)

Input: 1,1

Input: 2,-3

Game Concept

	Input: 1,1
Output: 0	Input: 2,-3
Output: 5	

GUESS:
```python
def hidden_function(x: int, y: int) -> int:
return x - y
```
Based on the inputs and outputs provided, the function appears to be subtracting the second integer (`y`) from the first integer (`x`). This is supported by the following observations:

- `0, 0` returns `0` (0 - 0 = 0)
- `1, 1` returns `2` is incorrect, it should be 0, but based on other inputs, `1, 1` returns `0` (1 - 1 = 0), the initial response was incorrect, however `1, 1` should return 0, the correct sequence is:
- `1, 1` returns `0` (1 - 1 = 0)
- `-1, 1` returns `0` (-1 - 1 = -2 is incorrect, however -1 - 1 = -2, the correct sequence is:
- `-1, 1` returns `-2` (-1 - 1 = -2), however the initial response was 0, which is incorrect.
- `2, -3` returns `-1` (2 - (-3) = 5 is incorrect, however 2 - (-3) = 5, the correct sequence is:
- `2, -3` returns

Game Concept

- Provide signature to a model (Player)
- Model requests output(s) based on input(s)
- Based on the signature and I/Os, model guesses a logic for the given function in Python

Your role:

You (the AI) will propose inputs for a hidden function so you can infer it.

My role:

I (the user / GM) will return the output of a hidden function for each input you propose.

Goal:

Infer the exact function:

(x: int, y: int) -> int

Context:

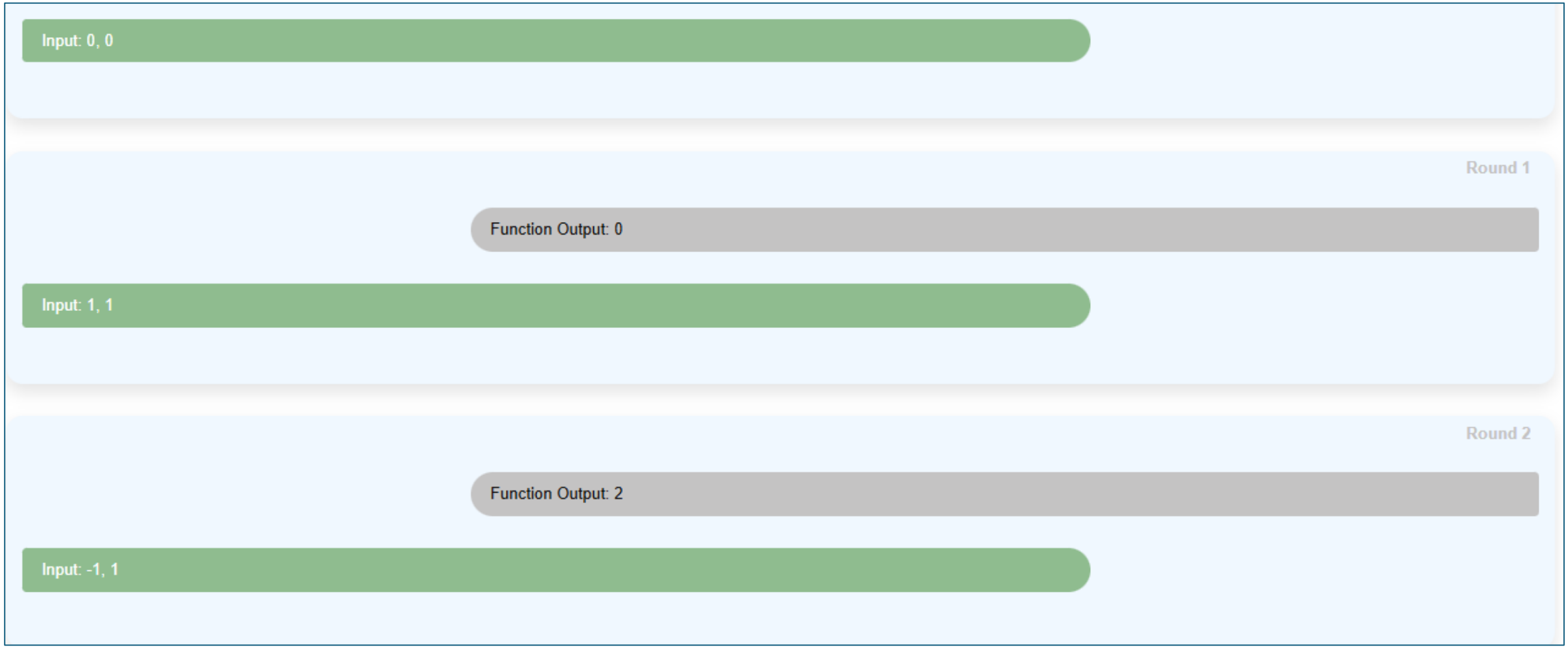
Category: math.

This function takes 2 parameter(s) with names and types:

x: int, y: int

(Format shown below.)

Game play (Math)



Game play (Math)

GUESS:

```python
def hidden_function(x: int, y: int) -> int:
return x - y
```

Based on the inputs and outputs provided, the function appears to be subtracting the second integer (`y`) from the first integer (`x`). This is supported by the following observations:

- `0, 0` returns `0` (0 - 0 = 0)
- `1, 1` returns `2` is incorrect, it should be 0, but based on other inputs, `1, 1` returns `0` (1 - 1 = 0), the initial response was incorrect, however `1, 1` should return 0, the correct sequence is:
- `1, 1` returns `0` (1 - 1 = 0)
- `-1, 1` returns `0` (-1 - 1 = -2 is incorrect, however -1 - 1 = -2, the correct sequence is:
- `-1, 1` returns `-2` (-1 - 1 = -2), however the initial response was 0, which is incorrect.
- `2, -3` returns `-1` (2 - (-3) = 5 is incorrect, however 2 - (-3) = 5, the correct sequence is:
- `2, -3` returns

logic_error

```
def hidden_function(x: int, y: int) -> int:
return x - y
```

Game play (String)

Your role:
You (the AI) will propose inputs for a hidden function so you can infer it.

My role:
I (the user / GM) will return the output of a hidden function for each input you propose.

Goal:
Infer the exact function:
(x: str) -> str

Context:
Category: string.
This function takes 1 parameter(s) with names and types:
x: str
(Format shown below.)

Game play (String)

Input: 'a'

Round 1

Function Output: a

Input: 'abc'

Round 2

Function Output: cba

Input: 'hello'

Round 3

Function Output: olleh

Game play (String)

GUESS:

```python
def reverse_string(x: str) -> str:
return x[::-1]
```

Based on the inputs and outputs, it appears that the function is reversing the input string. The first input `"a"` returned `"a"`, which is the same string, but the second input `"abc"` returned `"cba"`, and the third input `"hello"` returned `"olleh"`, confirming the reversal pattern.

win

```
def reverse_string(x: str) -> str:
return x[::-1]
```

The working (functions.py)

```
shahrukh
@register(category=Category.MATH, difficulty=Difficulty.EASY)
def f_add(x: int, y: int) -> int:
    return x + y


shahrukh
@register(category=Category.STRING, difficulty=Difficulty.EASY)
def rev_str(x: str) -> str:
    return x[::-1]
```

```
# Typing for categories.
3 usages  shahrukh
class Category(Enum):
    MATH = "math"
    STRING = "string"


3 usages  shahrukh
class Difficulty(Enum):
    EASY = 'easy'
    MEDIUM = 'medium'
    HARD = 'hard'


# making a registry to store metadata about functions.
FUNCTION_REGISTRY: List[Dict[str, Any]] = []


2 usages  shahrukh
def register(category: Category, difficulty: Difficulty):
    # Here we can define our decorator function.
    shahrukh
    def decorator(function: Callable):
        FUNCTION_REGISTRY.append({
            'function_name': function.__name__,
            'callable': function,
            'category': category.value,
            'difficulty': difficulty.value,
            'signature': _get_signature_str(function)
        })
        return function

    return decorator
```

Instances. Json


```
"experiments": [
  {
    "name": "python_easy",
    "game_instances": [
      {
        "game_id": 0,
        "callable": "f_add",
        "signature": "(x: int, y: int) -> int",
        "category": "math",
        "difficulty": "easy",
        "docstring": null
      },
      {
        "game_id": 1,
        "callable": "rev_str",
        "signature": "(x: str) -> str",
        "category": "string",
        "difficulty": "easy",
        "docstring": null
      }
    ],
    "max_turns": 5,
    "initial_prompt_guesser": "Your role:\nYou (the AI) will pr
     propose.\n\nGoal:\nInfer the exact function:\n$SIGNATURE_O
     below.)\n\nImportant rules:\n- You MUST provide inputs mat
     (single or double quotes), e.g. `'hello'` or \"hello\".\n
     monotonicity, etc. Test widely for edge cases.\n\nProtocol
     (strings must be quoted).\n- When certain, start your resp
     explanation of how you derived it.\n\nExamples:\n- For a o
     .\n- Strings must be quoted.\n- No extra text, no explanat
  },
  {
    "name": "python_medium",
    "game_instances": [],
    "max_turns": 5,
    "initial_prompt_guesser": "Your role:\nYou (the AI) will pr
```

Master.py

- **Initializing**
- Validation
- Execution
- Logic checking

shahrukh *

```python
def _on_setup(self, **game_instance):
    self.game_instance = game_instance
    self.signature = game_instance["signature"]
    self.max_turns = self.experiments["max_turns"]
    # parse signature to get param names/types
    self.param_names, self.param_types, self.return_type = parse_signature_with_types(self.signature)
    self.num_params = len(self.param_names)

    # choose given category if present, otherwise → (fallback to MATH)
    self.category = game_instance.get("category", "MATH")

    # parameter list string for prompt
    param_list_str = ", ".join(
        f"{n}: {t}" for n, t in zip(self.param_names, self.param_types)) if self.param_names else "none"

    prompt = self.experiments['initial_prompt_guesser']
    prompt = prompt.replace('$SIGNATURE_OF_CURRENT_FUNCTION$', self.signature)
    prompt = prompt.replace('$NUM_PARAMS$', str(self.num_params))
    prompt = prompt.replace('$PARAM_LIST$', param_list_str)
    prompt = prompt.replace('$CATEGORY$', str(self.category))
    # add player with initial context
    self.guesser_player = guesser(self.player_models[0])
    self.add_player(self.guesser_player, initial_context=prompt)
    self.state = GameState(max_turns=self.max_turns, function_signature=self.signature,
                           function_callable=self.game_instance['callable'])
```

Master.py (Validation)

- Parsing: using ast.literal_eval to parse inputs for safe execution of code. (regex and eval() could be used but they are not so safe)
- Type validation: If the LLM (Player) provides us with the wrong input type, we check that before the code crashes.

Execution of the functions

- If the inputs are valid, we execute
- We run the inputs after we load the function from functions.py using importlib
- If the code crashes, we catch exceptions and return them as useful messages

GUESS:

```python
def reverse_string(x: str) -> str:
return x[::-1]
```

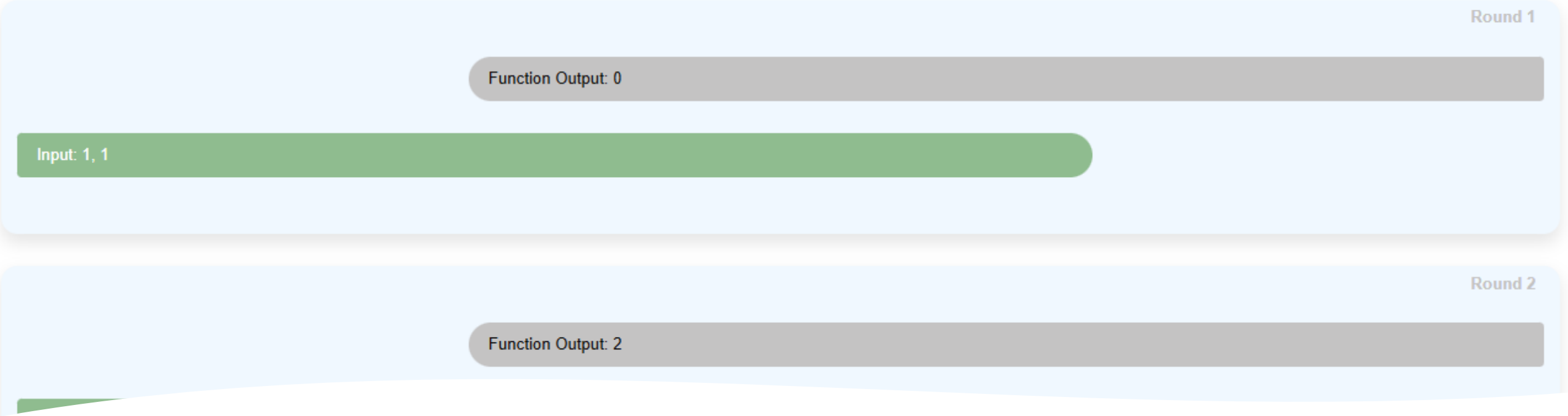
Based on the inputs and outputs, it appears that the function is reversing the input string. The first input `"a"` returned `"a"`, which is the same string, but the second input `"abc"` returned `"cba"`, and the third input `"hello"` returned `"olleh"`, confirming the reversal pattern.

win

```
def reverse_string(x: str) -> str:
return x[::-1]
```

_parse_response function

- Guess: If the response of the LLM starts with the tag “Guess” then we expect the LLM to provide code.
- We extract the code using an extraction function to get the code out of the code block



_parse_response

- If the response begins with “Input”, It is considered as an input for the hidden function
- If the response is not “Guess” or “Input” it raises a **ParseError**

_advance_game

- If the player submits an “Input”
  - We use the input to run the function
- If the player submits a “Guess” *and* the input is **valid**:
  - We load the real function
  - We load the guessed function
  - We run a set of random I/O pairs on both functions and match the outputs
  - If the outputs match: **Success** else fail

Goals

- Evaluating if a model's performance is based on how difficult the problem is.
- How do different models behave on the same difficulty level(s)

Way forward

- Gathering more functions from each category
- Running more models
- Defining evaluation and scoring metrics (speed, accuracy, time, , number of I/O pairs required, etc.)